

Memory Management Simulator
B.Tech Lab Project Report
on
Operating Systems Lab
By

Akula Tejashwini (24911A1203)

Under the Guidance of
Mrs.Ch. Swetha
Assistant Professor



DEPARTMENT OF INFORMATION TECHNOLOGY

VIDYA JYOTHI INSTITUTE OF TECHNOLOGY

(An Autonomous Institution)

**[Accredited by NAAC & NBA, Approved by AICTE New Delhi & Permanently
Affiliated to JNTUH]**

2025 – 2026



CERTIFICATE

This is to certify that the Operating Systems Lab project report entitled “**Memory Management Simulator**” submitted by AkulaTejashwini (24911A1203) to Vidya Jyothi Institute of Technology (An Autonomous Institution), Hyderabad, of B. Tech. II year in Information Technology a Bonafide record of Lab project work carried out by them under my supervision.

Signature of Supervisor

Mrs.Ch. Swetha
Assistant Professor

Signature of Head of the Department

Dr. A. Obulesu
Professor

Table of Contents

Chapter No	Title	Page.No
1	Introduction	4
2	System Requirement Specifications	5
	2.1 Software Requirements	
	2.2 Hardware Requirements	
3	System Architecture	6
4	Sample Executable Code	7
5	Result Screens	19
6	Conclusion	21

1.Introduction

The Memory Management Simulator is a web-based application designed to demonstrate and analyze different memory management techniques used in operating systems. Instead of manually calculating memory allocation and deallocation, this simulator provides a fast, visual, and automated solution.

The simulator supports:

- Multiple memory allocation algorithms (First Fit, Best Fit, Worst Fit)
- User input for memory block sizes and process sizes
- Automatic memory allocation
- Internal and external fragmentation analysis
- Memory block visualization
- Graph comparison of algorithm performance

This project demonstrates the use of core operating system concepts, memory allocation algorithms, and interactive UI design.

1.1 Purpose of the Project

- To simulate memory management algorithms in a visual and interactive way
- To understand memory allocation and fragmentation
- To compare performance of different allocation techniques
- To automate memory allocation calculations
- To demonstrate use of arrays, loops, and logic building in JavaScript
- To improve understanding of real-time OS memory management behavior

2.System Requirement Specifications

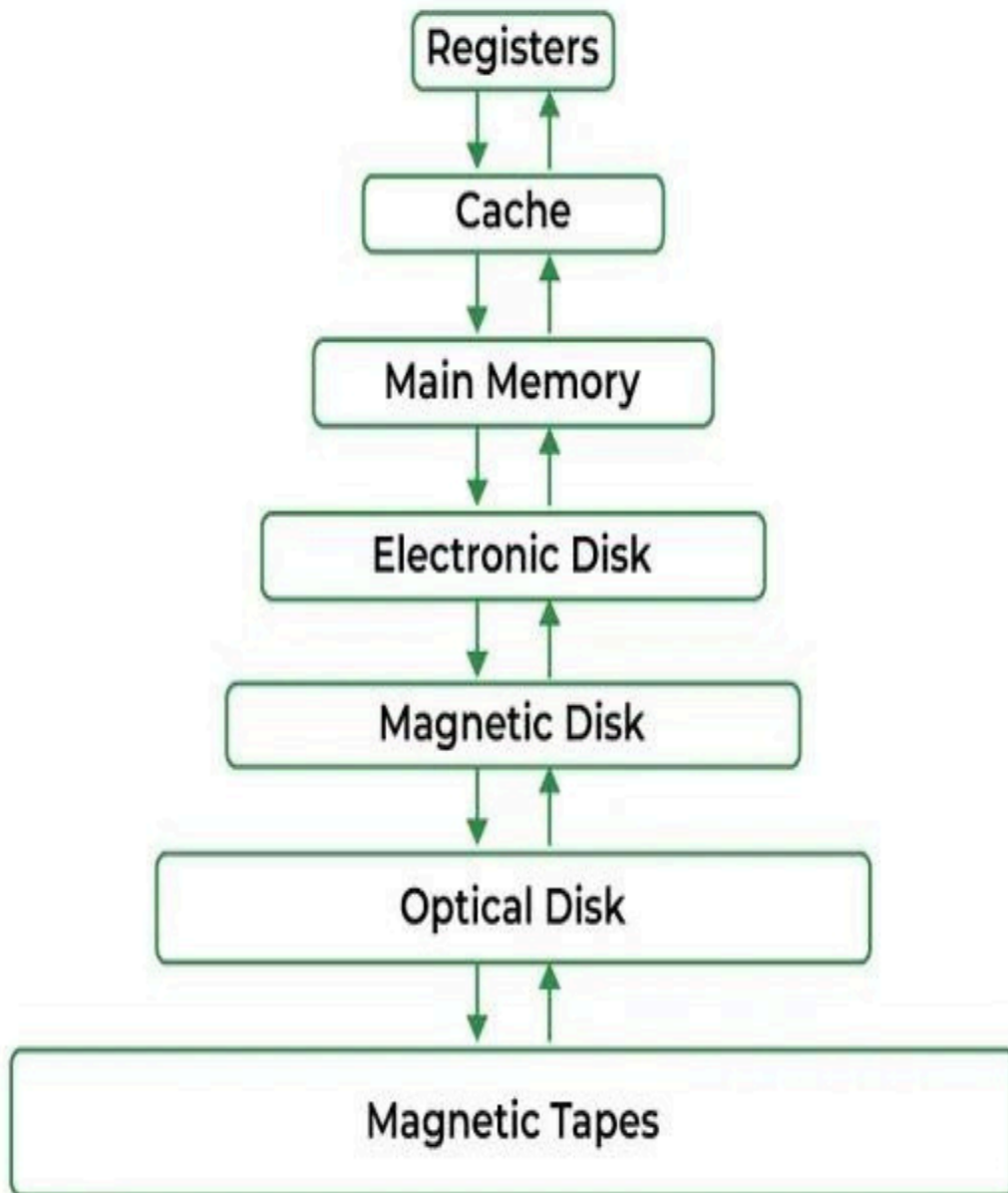
2.1 Hardware Requirements

- Processor: Minimum dual-core CPU
- RAM: Minimum 2 GB (Recommended 4 GB)
- Storage: At least 100 MB free
- Device: Laptop / Desktop / Mobile
- Display: Any standard resolution

2.2 Software Requirements

- Operating System: Windows / Linux / Android
- Platform: HTML, CSS, JavaScript
- Browser: Chrome / Edge / Firefox
- Database: Not required (runs locally)
- Tools Used:
 - VS Code / Any Code Editor
 - Web Browser

3. System Architecture



4.Sample Code

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Memory Management Simulator</title>

<script src="https://cdn.tailwindcss.com"></script>

<link href="https://cdn.jsdelivr.net/npm/font-awesome@6.0.0/css/all.min.css"
rel="stylesheet">

<style>

body {
font-family: 'Inter', sans-serif;
background-color: #f3f4f6;
}

/* Custom scrollbar for logs */

.log-container::-webkit-scrollbar {
width: 8px;
}

.log-container::-webkit-scrollbar-track {
background: #f1f1f1;
border-radius: 4px;
}

.log-container::-webkit-scrollbar-thumb {
background: #cbd5e1;
border-radius: 4px;
}
```

```

.log-container::-webkit-scrollbar-thumb:hover {
background: #94a3b8;
}
/* Animation for new blocks/processes */
@keyframes fadeInScale {
0% {
opacity: 0;
transform: scale(0.95);
}
100% {
opacity: 1;
transform: scale(1);
}
}
.animate-enter {
animation: fadeInScale 0.3s ease-out forwards;
}
/* Pulse animation for allocation */
@keyframes pulse-green {
0% {
box-shadow: 0 0 0 0 rgba(34, 197, 94, 0.7);
}
70% {
box-shadow: 0 0 0 10px rgba(34, 197, 94, 0);
}
100% {
box-shadow: 0 0 0 0 rgba(34, 197, 94, 0);
}
}.animate-allocate {
animation: pulse-green 1s ease-out;
}
</style>
</head>
<body class="min-h-screen text-gray-800 flex flex-col items-center py-8 px-4 sm:px-8">
<header class="w-full max-w-5xl mb-8 text-center">
<h1 class="text-4xl font-extrabold text-blue-900 mb-2 flex justify-center items-center gap-3">
<i class="fas fa-memory text-blue-600"></i>
Memory Management Simulator

```



```

</h1>
<p class="text-gray-600">
Visualize First Fit, Best Fit, and Worst Fit allocation algorithms.
</p></header>
<main class="w-full max-w-5xl grid grid-cols-1 lg:grid-cols-3 gap-8">
<!-- Left Column -->
<div class="lg:col-span-1 space-y-6">
<!-- Algorithm Card -->
<div class="bg-white rounded-xl shadow-md p-6 border-t-4 border-blue-500">
<h2 class="text-xl font-bold mb-4 flex items-center gap-2">
<i class="fas fa-cogs text-gray-500"></i>
Algorithm
</h2>
<div class="space-y-3">
<label class="flex items-center space-x-3 cursor-pointer group">
<input type="radio" name="algorithm" value="firstFit" checked
class="form-radio h-5 w-5 text-blue-600">
<span class="text-gray-700 font-medium group-hover:text-blue-600">
First Fit</span>
</label>
<label class="flex items-center space-x-3 cursor-pointer group">
<input type="radio" name="algorithm" value="bestFit"
class="form-radio h-5 w-5 text-blue-600">
<span class="text-gray-700 font-medium group-hover:text-blue-600">
Best Fit
</span>
</label>
<label class="flex items-center space-x-3 cursor-pointer group">
<input type="radio" name="algorithm" value="worstFit"
class="form-radio h-5 w-5 text-blue-600">
<span class="text-gray-700 font-medium group-hover:text-blue-600">
Worst Fit
</span>
</label></div>
</div><!-- Add Memory Block -->
<div class="bg-white rounded-xl shadow-md p-6 border-t-4 border-green-500">
<h2 class="text-xl font-bold mb-4 flex items-center gap-2">
<i class="fas fa-plus-square text-green-500"></i>

```

Add Memory Block

</h2>

<div class="flex flex-col space-y-3">

<div>

<label for="blockSize"

class="block text-sm font-medium text-gray-700 mb-1">

Block Size (KB)

</label>

<input type="number"

id="blockSize"

min="1"

placeholder="e.g., 100"

class="w-full px-4 py-2 border border-gray-300 rounded-md

focus:ring-green-500 focus:border-green-500 outline-none transition">

</div>

<button id="addBlockBtn"

class="w-full bg-green-600 hover:bg-green-700 text-white font-bold py-2 px-4 rounded-md transition duration-200 shadow-sm flex items-center justify-center gap-2">

<i class="fas fa-plus"></i>

Add Block

</button>

</div>

</div>

<!-- Add Process -->

<div class="bg-white rounded-xl shadow-md p-6 border-t-4 border-purple-500">

<h2 class="text-xl font-bold mb-4 flex items-center gap-2">

<i class="fas fa-microchip text-purple-500"></i>

Add Process

</h2>

<div class="flex flex-col space-y-3">

<div>

<label for="processSize"

class="block text-sm font-medium text-gray-700 mb-1">

Process Size (KB)

</label>

<input type="number"

id="processSize"

min="1"

placeholder="e.g., 50"

```

class="w-full px-4 py-2 border border-gray-300 rounded-md
focus:ring-purple-500 focus:border-purple-500 outline-none transition">
</div>
<button id="addProcessBtn"
class="w-full bg-purple-600 hover:bg-purple-700 text-white font-bold py-2 px-4 rounded-md
transition duration-200 shadow-sm flex items-center justify-center gap-2">
<i class="fas fa-play"></i>
Allocate Process
</button>
</div>
</div>
<!-- Reset -->
<div class="bg-white rounded-xl shadow-md p-6">
<button id="resetBtn"
class="w-full bg-red-500 hover:bg-red-600 text-white font-bold py-2 px-4 rounded-md transition
duration-200 shadow-sm flex items-center justify-center gap-2">
<i class="fas fa-trash-alt"></i>
Reset All
</button>
</div>
</div>
<!-- Right Column -->
<div class="lg:col-span-2 space-y-6">
<!-- Memory Display -->
<div class="bg-white rounded-xl shadow-md p-6 min-h-[400px] flex flex-col">
<div class="flex justify-between items-center mb-4 border-b pb-2">
<h2 class="text-2xl font-bold text-gray-800 flex items-center gap-2">
<i class="fas fa-hdd text-gray-600"></i>
Physical Memory
</h2>
</div>
<div id="memoryContainer"
class="flex-grow flex flex-col gap-3 overflow-y-auto pr-2 pb-2">
<div id="emptyState"
class="m-auto text-center text-gray-400 py-10">
<i class="fas fa-layer-group text-5xl mb-3 opacity-50"></i>
<p>No memory blocks added yet.</p>
<p class="text-sm">

```

Use the controls on the left to add blocks.</p>

</div></div></div>

<!-- Activity Log -->

<div class="bg-white rounded-xl shadow-md overflow-hidden flex flex-col" style="height: 250px;">

<div class="bg-gray-800 text-white px-4 py-3 flex justify-between items-center">

<h3 class="font-bold flex items-center gap-2">

<i class="fas fa-terminal"></i>

Activity Log</h3>

<button id="clearLogBtn"

class="text-xs text-gray-300 hover:text-white transition">

Clear</button>

</div>

<div id="logContainer"

class="p-4 bg-gray-50 flex-grow overflow-y-auto log-container text-sm font-mono space-y-1">

</div></div></div>

</main>

<script>

let memoryBlocks = [];

let processCount = 0;

let blockCount = 0;

// DOM Elements

const blockSizeInput = document.getElementById('blockSize');

const addBlockBtn = document.getElementById('addBlockBtn');

const processSizeInput = document.getElementById('processSize');

const addProcessBtn = document.getElementById('addProcessBtn');

const resetBtn = document.getElementById('resetBtn');

const memoryContainer = document.getElementById('memoryContainer');

const emptyState = document.getElementById('emptyState');

const logContainer = document.getElementById('logContainer');

const clearLogBtn = document.getElementById('clearLogBtn');

// Add Log

function addLog(message, type = 'info') {

const time = new Date().toLocaleTimeString([], { hour12: false });

const logEl = document.createElement('div');

let colorClass = 'text-gray-700';

if (type === 'success')

colorClass = 'text-green-600 font-semibold';

if (type === 'error')

colorClass = 'text-red-500 font-semibold';

if (type === 'warning')colorClass = 'text-orange-500 font-semibold';

```

if (type === 'system')
colorClass = 'text-blue-600 italic';
logEl.innerHTML = `
<span class="text-gray-400 mr-2">[${time}]</span>
<span class="${colorClass}">${message}</span>`;
logContainer.appendChild(logEl);
logContainer.scrollTop = logContainer.scrollHeight;
}
// Selected Algorithm
function getAlgorithm() {
return document.querySelector(
'input[name="algorithm"]:checked'
).value;
}
// Render Memory
function renderMemory() {
if (memoryBlocks.length === 0) {
emptyState.style.display = 'block';
memoryContainer.innerHTML = "";
memoryContainer.appendChild(emptyState);
return;
}
emptyState.style.display = 'none';
memoryContainer.innerHTML = "";
memoryBlocks.forEach((block, index) => {
const isAllocated = block.process !== null;
const utilization = isAllocated
? (block.process.size / block.size) * 100
: 0;
const blockEl = document.createElement('div');
blockEl.className = `
relative rounded-lg border-2 overflow-hidden animate-enter transition-all duration-300${isAllocated
? 'border-blue-400 bg-blue-50'
: 'border-gray-300 bg-gray-100 hover:border-gray-400'}
${block.justAllocated ? 'animate-allocate' : ''}`;
if(block.justAllocated)
block.justAllocated = false;

```

```

const bgStyle = isAllocated
? `background: linear-gradient(90deg,
  rgba(147, 197, 253, 0.5) ${utilization}%,
  transparent ${utilization}%)`;
: "";
blockEl.innerHTML = `
<div class="absolute inset-0 z-0"
style="${bgStyle}"></div>
<div class="relative z-10 p-3 flex justify-between items-center">
<div>
<div class="font-bold text-gray-800">
Block ${block.id}
<span class="text-xs font-normal text-gray-500 ml-1">
(${block.size} KB)
</span>
</div>
<div class="text-sm mt-1">
${isAllocated
? `<span class="inline-flex items-center gap-1 text-purple-700 font-semibold bg-purple-100 px-2 py-0.5 rounded">
<i class="fas fa-microchip text-xs"></i>
Process ${block.process.id}
(${block.process.size} KB)
</span>`
: `<span class="text-green-600 font-medium flex items-center gap-1">
<i class="fas fa-check-circle text-xs"></i>
Free
</span>`}
}
</div>
</div>
<div class="flex flex-col items-end">
<div class="text-xs font-mono bg-white px-2 py-1 rounded shadow-sm border border-gray-200 mb-1">
Fragment:
<span class="${
isAllocated && (block.size - block.process.size) > 0
? 'text-red-500 font-bold'
: 'text-gray-600'
}">

```

```

    ${isAllocated
    ? block.size - block.process.size
    : block.size}
    KB</span></div>
    ${isAllocated
    ? `<button onclick="deallocateProcess(${index})"
    class="text-xs bg-red-100 text-red-600 hover:bg-red-200 px-2 py-1 rounded transition border border-
    red-200">
    Free Memory
    </button>`
    : `<button onclick="removeBlock(${index})"
    class="text-xs text-gray-400 hover:text-red-500 px-2 py-1 rounded transition">
    <i class="fas fa-times"></i>
    </button>`}
    </div></div>`;
    memoryContainer.appendChild(blockEl);
    });}
    // Add Memory Block
    addBlockBtn.addEventListener('click', () => {
    const size = parseInt(blockSizeInput.value);
    if (isNaN(size) || size <= 0) {
    addLog(
    "Invalid block size. Please enter a positive number.",
    "error");
    return;}
    blockCount++;
    memoryBlocks.push({
    id: blockCount,
    size: size,
    process: null});
    addLog(
    `Added memory Block ${blockCount} of size ${size} KB.`,
    "system");
    blockSizeInput.value = "";
    renderMemory();});
    // Remove Block
    window.removeBlock = function(index) {
    const block = memoryBlocks[index];
    if (block.process !== null) {
    addLog(`Cannot remove Block ${block.id}; it is currently in use.`, "error");
    return;}

```

```

}if (memoryBlocks.length === 0) {
addLog(
"No memory blocks available. Please add blocks first.",
"warning");
return;}
processCount++;
const newProcess = {
id: processCount,
size: size};
const algo = getAlgorithm();
let allocatedIndex = -1;
addLog(`Attempting to allocate Process ${newProcess.id} (${size} KB) using ${algo}...`);
// First Fit
if (algo === 'firstFit') {
for (let i = 0; i < memoryBlocks.length; i++) {
if (memoryBlocks[i].process === null &&memoryBlocks[i].size >= size) {
allocatedIndex = i;
break;}}}}
// Best Fit
else if (algo === 'bestFit') {
let minDiff = Infinity;
for (let i = 0; i < memoryBlocks.length; i++) {
if (
memoryBlocks[i].process === null &&
memoryBlocks[i].size >= size) {
let diff = memoryBlocks[i].size - size;
if (diff < minDiff) {
minDiff = diff;
allocatedIndex = i;}}}}
// Worst Fit
else if (algo === 'worstFit') {
let maxDiff = -1;
for (let i = 0; i < memoryBlocks.length; i++) {
if (memoryBlocks[i].process === null &&memoryBlocks[i].size >= size
) {
let diff = memoryBlocks[i].size - size;
if (diff > maxDiff) {
maxDiff = diff;
allocatedIndex = i;
}}
}}

```



```

// Allocation
if (allocatedIndex !== -1) {
memoryBlocks[allocatedIndex].process = newProcess;
memoryBlocks[allocatedIndex].justAllocated = true;
const fragment = memoryBlocks[allocatedIndex].size - size;
addLog(`Success:      Process      ${newProcess.id}      allocated      to      Block
${memoryBlocks[allocatedIndex].id}. Internal Fragmentation: ${fragment} KB.`,
"success");
processSizeInput.value = "";
renderMemory();
} else {
addLog(
`Failed: Not enough contiguous memory for Process ${newProcess.id} (${size} KB).`,
"error");
}
});
// Deallocate
window.deallocateProcess = function(blockIndex) {
const block = memoryBlocks[blockIndex];
const pId = block.process.id;
block.process = null;
addLog(
`Freed memory from Block ${block.id}. Process ${pId} terminated.`,
"info"
);
renderMemory();
};
// Reset
resetBtn.addEventListener('click', () => {
memoryBlocks = [];
processCount = 0;
blockCount = 0;
addLog(
"System reset. All memory blocks and processes cleared.",
"system"
);
renderMemory();
});

```

```

// Clear Logs
clearLogBtn.addEventListener('click', () => {
  logContainer.innerHTML = "";
});
// Enter Key
blockSizeInput.addEventListener('keypress', function (e) {
  if (e.key === 'Enter')
    addBlockBtn.click();
});
processSizeInput.addEventListener('keypress', function (e) {
  if (e.key === 'Enter')
    addProcessBtn.click();
});
// Initial Setup
addLog(
  "Simulator initialized. Select an algorithm and add memory blocks to begin.",
  "system"
);
// Default Blocks
setTimeout(() => {
  blockSizeInput.value = 100;
  addBlockBtn.click();
  blockSizeInput.value = 500;
  addBlockBtn.click();
  blockSizeInput.value = 200;
  addBlockBtn.click();
  blockSizeInput.value = 300;
  addBlockBtn.click();
  blockSizeInput.value = 600;
  addBlockBtn.click();
  processSizeInput.value = 212;
}, 500);
</script>
</body>
</html>

```

5.ResultScreen

Memory Management Simulator

Visualize First Fit, Best Fit, and Worst Fit allocation algorithms.

Algorithm

☒ First Fit

☐ Best Fit

☐ Worst Fit

+ Add Memory Block

Block Size (KB)

e.g., 100

+ Add Block

+ Add Process

Process Size (KB)

212

▶ Allocate Process

Reset All

Physical Memory

Free Allocated

Block 1 (100 KB)

Fragment: 100 KB

Free

×

Block 2 (500 KB)

Fragment: 500 KB

Free

×

Block 3 (200 KB)

Fragment: 200 KB

Free

×

Block 4 (300 KB)

Fragment: 300 KB

Free

×

Block 5 (600 KB)

Fragment: 600 KB

Free

×

Block 6 (1000 KB)

Fragment: 1000 KB

Free

×

> Activity Log

Clear

[19:48:40] Simulator initialized. Select an algorithm and add memory blocks to begin.

[19:48:41] Added memory Block 1 of size 100 KB.

[19:48:41] Added memory Block 2 of size 500 KB.

[19:48:41] Added memory Block 3 of size 200 KB.

[19:48:41] Added memory Block 4 of size 300 KB.

[19:48:41] Added memory Block 5 of size 600 KB.

[19:49:12] Added memory Block 6 of size 1000 KB.

Memory Management Simulator

Visualize First Fit, Best Fit, and Worst Fit allocation algorithms.

Algorithm

- ☐ First Fit
- ☒ Best Fit
- ☐ Worst Fit

Add Memory Block

Block Size (KB)

e.g., 100

 + Add Block

Add Process

Process Size (KB)

e.g., 50

 ▶ Allocate Process

 Reset All

Physical Memory

☐ Free ☒ Allocated

Block 1 (100 KB)

Fragment: 100 KB

✔ Free

Block 2 (500 KB)

Fragment: 500 KB

✔ Free

Block 3 (200 KB)

Fragment: 200 KB

✔ Free

Block 4 (300 KB)

Fragment: 88 KB

 Process 1 (212 KB)

Free Memory

Block 5 (600 KB)

Fragment: 600 KB

✔ Free

Block 6 (1000 KB)

Fragment: 1000 KB

✔ Free

Block 7 (122 KB)

Fragment: 122 KB

✔ Free

>_ Activity Log

Clear

[19:48:41] Added memory Block 3 of size 200 KB.
[19:48:41] Added memory Block 4 of size 300 KB.
[19:48:41] Added memory Block 5 of size 600 KB.
[19:49:12] Added memory Block 6 of size 1000 KB.
[19:51:34] Added memory Block 7 of size 122 KB.
[19:51:39] Attempting to allocate Process 1 (212 KB) using bestFit...
[19:51:39] Success: Process 1 allocated to Block 4. Internal Fragmentation: 88 KB.

6.Conclusion

TheMemoryManagement Simulator successfully demonstrates how memory allocation algorithm works in an operating system. It provides a clear and interactive way to understand memory allocation, fragmentation, and process management.

This project showcases:

- Implementation of memory allocation algorithms
- Use of JavaScript logic and arrays
- Dynamic UI generation
- Visualization of memory allocation

Overall, the simulator achieves its objective of simplifying complex memory management concepts. With further improvements such as Paging, Segmentation, and Virtual Memory simulation, the system can be enhanced into a more advanced learning tool.